UNIVERSIDADE POLITÉCNICA DO CÁVADO E DO AVE

ESCOLA SUPERIOR DE TECNOLOGIA IPCA

TunaEngine Plataforma de Gestão de Tunas

Relatório técnico de arquitetura e implementação

Arquiteturas e Integração de Sistemas

Diogo Caldas - 27951

Guilherme de Jesus - 28043

Mateus Costa - 22845

Ano letivo 2026/2027

Mestrado em Engenharia Informática

Escola Superior de Tecnologia

Universidade Politécnica do Cávado e do Ave

RESUMO

Este relatório descreve o protótipo TunaEngine existente na pasta CSharp. O backend é composto por quatro APIs ASP.NET Core independentes - Members, Events, Documents e Inventory - e por um API Gateway implementado com YARP. O cliente .NET MAUI envia pedidos HTTP/JSON ao Gateway, que encaminha cada rota para o serviço correspondente. Os serviços persistem dados em bases MongoDB separadas; Members também gere cargos e Documents gere tipos documentais. O código inclui Dockerfiles e instruções de publicação dos cinco serviços no Render. Foi acrescentada a especificação estática CSharp/openapi.yaml, baseada nas rotas, DTOs e respostas atuais; a aplicação não a gera nem serve em runtime. Também não se encontrou autenticação e autorização por token, integração com um serviço externo de negócio, projeto de testes automatizados ou resultados de execução documentados. A collection Postman contém 15 pedidos usados pelo cliente, sem evidência de execução. O protótipo demonstra separação de responsabilidades, encaminhamento e persistência por domínio; segurança e validação continuam incompletas.

Palavras-chave: microserviços, API Gateway, ASP.NET Core, .NET MAUI, MongoDB, YARP

ABSTRACT

This report describes the TunaEngine prototype in the CSharp folder. Its backend contains four independent ASP.NET Core APIs - Members, Events, Documents, and Inventory - plus a YARP API Gateway. The .NET MAUI client sends HTTP/JSON requests to the Gateway, which forwards each route to its service. The services persist data in separate MongoDB databases; Members also manages roles, and Documents manages document types. The code includes Dockerfiles and deployment instructions for five Render services. This review adds the static OpenAPI specification CSharp/openapi.yaml, based on the current routes, DTOs, and responses; the application does not generate or serve it at runtime. The APIs still lack token-based authentication and authorization, external business-service integration, an automated test project, and documented execution results. The Postman collection contains 15 requests used by the client, but there is no evidence that it was run. API security and execution validation remain incomplete.

Keywords: microservices, API Gateway, ASP.NET Core, .NET MAUI, MongoDB, YARP

ÍNDICE

1. Introdução	7
1.1 Objetivos	7
1.2 Contexto e âmbito	7
1.3 Estrutura do relatório	7
2. Problema e requisitos	8
2.1 Utilizadores e processo principal	8
2.2 Requisitos priorizados	8
2.3 Cobertura da implementação	9
3. Arquitetura baseada em microserviços	10
3.1 Visão geral	10
3.2 Serviços e responsabilidades	11
3.3 Comunicação e segurança	11
3.4 Dados e persistência	11
4. Implementação	12
4.1 Tecnologias	12
4.2 Members e cargos	12
4.3 Events	12
4.4 Documents	12
4.5 Inventory	12
4.6 Gateway, cliente e contrato da API	12
5. Instalação e configuração	14
5.1 Pré-requisitos	14
5.2 Execução local	14
5.3 Configuração, Render e aplicação cliente	14
6. Validação e resultados	15
6.1 Cobertura da collection Postman	15
6.2 Evidência e limites da validação	15
7. Limitações e trabalho futuro	16
8. Conclusão	17
Bibliografia	17

ÍNDICE DE FIGURAS

Figura 1: Arquitetura atual da TunaEngine	10

ÍNDICE DE TABELAS

Tabela 1: Síntese do âmbito funcional implementado	8
Tabela 2: Cobertura dos requisitos do enunciado	9
Tabela 3: Serviços e responsabilidades	11
Tabela 4: Endpoints principais	13
Tabela 5: Configuração dos containers e portas	14

Glossário

API: interface HTTP através da qual uma aplicação disponibiliza operações a outros programas.

API Gateway: ponto de entrada que encaminha pedidos para os serviços responsáveis.

ASP.NET Core: plataforma da Microsoft usada para criar aplicações e APIs web.

Microserviço: aplicação autónoma responsável por um domínio funcional delimitado.

MongoDB: sistema de gestão de bases de dados orientado a documentos.

OpenAPI: standard para descrever operações, parâmetros, respostas e esquemas de uma API HTTP.

REST: estilo de comunicação sobre HTTP que organiza operações com métodos como GET, POST, PUT e DELETE.

Tuna: grupo musical de estudantes universitários que cultiva a música tradicional e o convívio académico.

Siglas e Acrónimos

API: Application Programming Interface

ASP.NET Core: plataforma de desenvolvimento de aplicações web da Microsoft

C#: linguagem de programação usada nos projetos

CRUD: Create, Read, Update and Delete

HTTP: Hypertext Transfer Protocol

JSON: JavaScript Object Notation

MAUI: .NET Multi-platform App UI

MongoDB: sistema de bases de dados orientado a documentos

YARP: Yet Another Reverse Proxy

1. Introdução

A organização de uma tuna envolve membros, direção, eventos, documentos e material. A dispersão desta informação dificulta a consulta e a atualização dos registos. A TunaEngine reúne estes domínios numa aplicação .NET MAUI e num conjunto de APIs ASP.NET Core. A implementação analisada é a que está atualmente na pasta CSharp, incluindo o Gateway e quatro serviços de negócio; esta versão difere da descrita no relatório recebido [1, 2].

1.1 Objetivos

O objetivo do trabalho é analisar um processo de gestão de tuna e construir um protótipo distribuído que demonstre serviços REST, persistência por domínio, containers e integração dos componentes. O relatório distingue as funcionalidades existentes das que ainda não satisfazem o enunciado, sem atribuir ao sistema comportamentos que não estão no código [1, 2].

1.2 Contexto e âmbito

O público-alvo previsto inclui membros e elementos da direção. O cliente atual permite iniciar sessão, consultar e gerir membros, eventos, documentos e inventário. A API de login verifica credenciais, mas não emite token; os pedidos seguintes não são associados a uma identidade autenticada no servidor. Os dados de cargos existem, mas não são aplicados como políticas de autorização.

1.3 Estrutura do relatório

O capítulo 2 apresenta o problema e a cobertura dos requisitos. O capítulo 3 descreve o Gateway, os serviços, a comunicação e os dados. O capítulo 4 resume as APIs e o cliente. O capítulo 5 explica a execução local e a configuração Render. O capítulo 6 regista a evidência disponível e os limites da validação. Os capítulos 7 e 8 apresentam limitações e conclusões, seguidos das referências.

2. Problema e requisitos

2.1 Utilizadores e processo principal

No fluxo disponível, o utilizador inicia sessão e o cliente consulta dados através do Gateway. O membro pode consultar eventos, inventário e documentos; a interface também permite operações sobre membros e artigos. A direção pode criar eventos e gerir registos. O código não implementa notificações, uma ação autónoma de confirmação de presença ou uma checklist de tarefas.

O enunciado pede uma solução distribuída com API especificada em OpenAPI, integração entre serviços, segurança, serviço externo, testes documentados, containers e exploração da Cloud [1]. A tabela seguinte separa o que existe na pasta CSharp do que permanece por implementar ou validar.

2.2 Requisitos priorizados

Área	Requisito ou função	Estado no protótipo atual
Acesso	Iniciar sessão e proteger operações.	Login e hash de palavra-passe existem; não há token nem autorização nas APIs.
Membros e cargos	Gerir membros, cargos e permissões.	CRUD de membros e cargos; não há políticas por cargo e a atualização de membro é parcial.
Eventos	Criar, consultar e manter eventos e presenças.	CRUD de eventos; sem endpoint dedicado de presença ou checklist na interface.
Inventário	Consultar e gerir material.	CRUD implementado na API e na interface principal.
Documentos	Gerir documentos e tipos documentais.	CRUD de metadados e tipos; não há upload de ficheiros.
Gateway	Integrar os serviços e expor uma entrada comum.	YARP encaminha rotas para quatro serviços; não há chamadas entre serviços de negócio.
OpenAPI	CSharp/openapi.yaml documenta rotas, contratos JSON e respostas; ficheiro estático, sem endpoint runtime.	Documentado em ficheiro; sem geração/serviço automático.
Serviço externo	Integrar pelo menos um fornecedor externo de negócio.	Não foi encontrada integração externa de negócio.
Validação	Realizar e documentar testes e validações.	Collection de 15 pedidos disponível; sem execução documentada ou projeto de testes.
Cloud e containers	Isolar serviços em containers e explorar a Cloud.	Dockerfiles e instruções Render existem; estado operacional não confirmado.

Tabela 1: Síntese do âmbito funcional implementado

A solução já demonstra serviços ASP.NET Core independentes, encaminhamento por Gateway, CRUD sobre MongoDB, endpoints de saúde, Dockerfiles e configuração para Render. A especificação estática CSharp/openapi.yaml, acrescentada nesta revisão, documenta as rotas e os contratos atuais. A aplicação não gera nem serve OpenAPI em runtime. Também não se encontrou autorização por token, integração com um fornecedor externo de negócio ou projeto de testes automatizados. A collection Postman contém definições de pedidos, mas não resultados do Collection Runner.

2.3 Cobertura da implementação

Requisito do enunciado	Evidência na pasta CSharp	Avaliação
Arquitetura distribuída	Quatro APIs ASP.NET Core, Gateway YARP e cliente MAUI.	Implementado no código.
API REST	Controladores com operações HTTP para membros, cargos, eventos, documentos e inventário.	Implementado; sem prefixo de versão.
OpenAPI	Foi acrescentado CSharp/openapi.yaml; sem AddOpenApi/MapOpenApi ou endpoint no Gateway.	Documentado estaticamente; falta geração/serviço runtime.
Integração e segurança	O Gateway encaminha pedidos; login verifica credenciais e gera hash de palavra-passe.	Parcial: sem chamadas entre serviços e sem autorização por token.
Serviço externo	Não foi encontrado cliente para um fornecedor externo de negócio.	Por cumprir.
Testes documentados	Collection Postman com 15 pedidos; nenhum resultado de execução ou projeto de testes localizado.	Parcial; falta executar e registar evidência.
Cloud e containers	Dockerfiles para cinco serviços, URLs Render e guia de configuração.	Configuração presente; disponibilidade e operação não confirmadas.

Tabela 2: Cobertura dos requisitos do enunciado

3. Arquitetura baseada em microserviços

3.1 Visão geral

A arquitetura atual inclui o cliente .NET MAUI, um Gateway baseado em YARP e quatro APIs ASP.NET Core. O cliente usa a base https://tunaengine-gateway.onrender.com/; o Gateway encaminha /api/members e /api/roles para Members, /api/events para Events, /api/documents e /api/doc-types para Documents, e /api/inventory para Inventory. Cada API tem um Dockerfile e um endpoint /health. A Figura 1 representa estas ligações [2, 4].

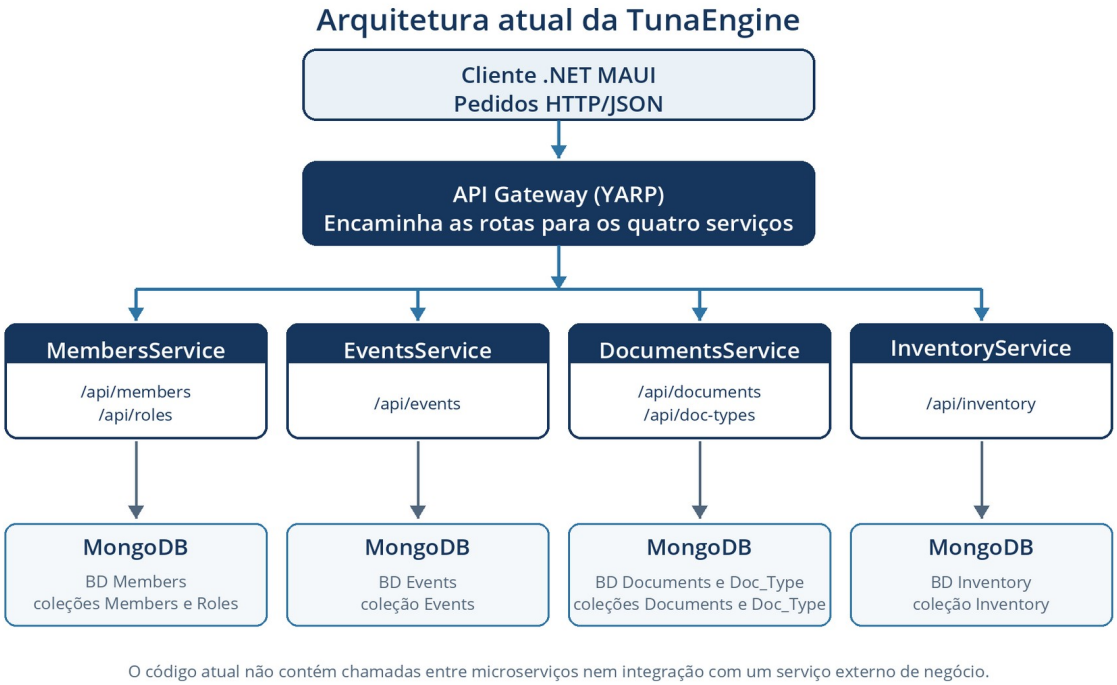


Figura 1: Arquitetura atual da TunaEngine

Cada serviço liga-se à sua base lógica MongoDB por meio do MongoDB.Driver. O Gateway não acede à base de dados e não executa lógica de negócio. A inspeção do código não encontrou chamadas REST entre Events e Inventory ou entre os restantes microserviços. Assim, a integração implementada ocorre entre o cliente e as APIs através do Gateway, não entre os serviços de negócio.

3.2 Serviços e responsabilidades

Serviço (porta)	Responsabilidade e rotas	Base lógica MongoDB
Members (10001)	Login, membros e cargos; /api/members, /api/roles	Members (coleções Members e Roles)
Events (10002)	CRUD de eventos; /api/events	Events
Inventory (10003)	CRUD de inventário; /api/inventory	Inventory
Documents (10004)	CRUD de documentos e tipos; /api/documents, /api/doc-types	Documents e Doc_Type
Gateway (10000)	Encaminha rotas para os quatro serviços; /health	Não usa MongoDB

Tabela 3: Serviços e responsabilidades

3.3 Comunicação e segurança

O cliente e a collection Postman enviam pedidos HTTP com corpos JSON ao Gateway. O YARP encaminha cada rota para o serviço configurado; não altera o modelo de autenticação porque o código não implementa tokens Bearer nem validação de identidade [2-4].

A API Members expõe registo, login e operações CRUD em /api/members. A mesma API disponibiliza CRUD em /api/roles. No registo, a palavra-passe é armazenada com o PasswordHasher do ASP.NET Identity; o login verifica o hash e migra para hash credenciais antigas guardadas em texto. Os outros serviços não exigem autenticação e nenhum controlador aplica autorização por cargo. A existência de um RoleId nos dados não constitui uma política de acesso.

As URLs públicas definidas no cliente e no Gateway usam HTTPS no Render; os containers escutam HTTP na porta fornecida pela plataforma. O código não configura autenticação entre cliente e serviços, gestão de permissões, limitação de pedidos ou gestão de segredos além da variável MongoDb__ConnectionString. Também não foi encontrada integração com fornecedor externo de negócio: MongoDB e Render fornecem persistência e alojamento [1, 2].

3.4 Dados e persistência

O MongoDB é partilhado como infraestrutura, mas os serviços usam bases lógicas distintas: Members usa a base Members com as coleções Members e Roles; Events usa Events; Documents usa Documents e Doc_Type; Inventory usa Inventory. Cada serviço mantém ainda a sua coleção de sequências para gerar códigos de negócio. Esta separação é configurável por MongoDbSettings [2, 5].

A separação não implementa multi-tenancy: os modelos não associam registos a uma tuna nem filtram por organização ou cargo. Não há base de utilizadores separada nem serviço dedicado à gestão de tunas.

4. Implementação

4.1 Tecnologias

Os serviços usam C# e ASP.NET Core 10 com controladores REST e MongoDB.Driver 3.12.0. O Gateway usa YARP 2.3.0. O cliente é uma aplicação .NET MAUI multi-alvo. O ficheiro CSharp/openapi.yaml documenta estaticamente as rotas e os esquemas atuais; a aplicação não gera nem serve a especificação em runtime. A presença dos pacotes não equivale a implementar autenticação ou autorização [2-7].

4.2 Members e cargos

Members expõe login em POST /api/members/login e operações CRUD em /api/members. A mesma API disponibiliza CRUD em /api/roles. As palavras-passe novas são guardadas com hash, mas o login devolve apenas id, nome, email e roleId: não há JWT. O código de atualização de membro persiste nome, email, telefone, tipo de voz e, se fornecida, uma nova palavra-passe; não persiste tunaName, studentNumber, roleId ou instrument, embora estes campos sejam aceites no pedido. Esta diferença deve ser considerada ao avaliar a funcionalidade.

4.3 Events

Events disponibiliza leitura, criação, atualização e remoção de eventos, com validação dos campos principais e das quantidades. O modelo inclui participantes, instrumentos necessários e boleias embutidos no evento. Não existe rota dedicada para registar presença; o cliente principal lista e cria eventos, sem expor operações de atualização ou remoção na interface.

4.4 Documents

Documents disponibiliza CRUD de documentos e de tipos documentais. Os documentos guardam metadados, data, observações e um campo File_Path. O serviço não recebe nem armazena ficheiros binários. O cliente principal consulta e cria registos documentais, mas não usa a API de tipos de documento diretamente.

4.5 Inventory

Inventory disponibiliza CRUD de artigos, valida identificadores MongoDB e rejeita designações ou marcas vazias e preços negativos. O cliente principal consulta, cria, atualiza e remove artigos. Os códigos de inventário são gerados pelo serviço.

4.6 Gateway, cliente e contrato da API

O cliente envia pedidos por TunaEngineApiClient ao Gateway. A collection CSharp/TunaEngine.postman_collection.json reúne 15 chamadas do cliente. A especificação CSharp/openapi.yaml documenta as rotas e os esquemas atuais; é estática e não tem endpoint no Gateway [2, 7].

API	Endpoints principais
Members	GET/POST /api/members; GET/PUT/DELETE /api/members/{id}; POST /api/members/login
Roles	GET/POST /api/roles; GET/PUT/DELETE /api/roles/{id}
Events	GET/POST /api/events; GET/PUT/DELETE /api/events/{id}
Documents	GET/POST /api/documents; GET/PUT/DELETE /api/documents/{id}
Document types	GET/POST /api/doc-types; GET/PUT/DELETE /api/doc-types/{id}
Inventory e saúde	CRUD /api/inventory e /api/inventory/{id}; GET /health em cada serviço e no Gateway

Tabela 4: Endpoints principais

O enunciado exige documentação da API segundo OpenAPI [1]. Esta revisão acrescenta CSharp/openapi.yaml, uma especificação estática das rotas, contratos JSON e respostas identificados nos controladores e DTOs. A aplicação não gera nem serve a especificação em runtime. A collection Postman é útil para chamadas manuais, mas não substitui o contrato OpenAPI [2, 7].

5. Instalação e configuração

5.1 Pré-requisitos

Para execução direta são necessários o SDK .NET 10 e acesso a uma instância MongoDB. Para construir e executar as imagens são necessários Docker e acesso às imagens base do .NET. Não existe um ficheiro docker-compose na pasta CSharp; cada processo ou container tem de ser iniciado separadamente.

5.2 Execução local

A partir da pasta CSharp, definir MongoDb__ConnectionString no ambiente de cada um dos quatro serviços de dados e iniciar cada aplicação num terminal separado. Os projetos podem ser executados com os comandos seguintes; o Gateway não requer acesso a MongoDB.

```
dotnet run --project .\TunaEngine.MembersService
dotnet run --project .\TunaEngine.EventsService
dotnet run --project .\TunaEngine.DocumentsService
dotnet run --project .\TunaEngine.InventoryService
dotnet run --project .\TunaEngine.Gateway
```

Cada Dockerfile publica uma imagem baseada em .NET 10 e define uma porta de fallback: Gateway 10000, Members 10001, Events 10002, Inventory 10003 e Documents 10004. No Render, o processo usa a variável PORT da plataforma. As rotas de saúde são GET /health. A tabela 5 resume as portas de container e verificações [2].

Serviço/container	Porta padrão	Verificação
Gateway	10000	GET /health; encaminha pedidos /api/*
MembersService	10001	GET /health
EventsService	10002	GET /health
InventoryService	10003	GET /health
DocumentsService	10004	GET /health

Tabela 5: Configuração dos containers e portas

5.3 Configuração, Render e aplicação cliente

A documentação MICROSERVICES_RENDER.md descreve a criação de cinco Web Services Docker no Render. Os quatro serviços de negócio precisam de MongoDb__ConnectionString; o Gateway encaminha as rotas para os domínios configurados. O cliente aponta para https://tunaengine-gateway.onrender.com/. O código e a documentação mostram uma configuração de publicação, mas a disponibilidade atual dos cinco serviços não foi confirmada nesta revisão.

O protótipo não usa JWT; MongoDB é configurado por variável de ambiente. A pasta não documenta Compose, gestão de segredos, rollback ou escalabilidade. O cliente MAUI suporta Android, iOS, Mac Catalyst e Windows [2, 6].

6. Validação e resultados

6.1 Cobertura da collection Postman

A collection CSharp/TunaEngine.postman_collection.json contém 15 definições de pedido: login; consulta, detalhe, criação, atualização e remoção de membros; listagem de cargos; consulta e criação de eventos e documentos; e CRUD de inventário. As operações passam pelo Gateway e usam variáveis para a URL, identificadores e corpos. A collection não inclui todas as rotas existentes nos serviços, como CRUD de cargos e tipos documentais, e não há resultados do Collection Runner associados.

6.2 Evidência e limites da validação

A validação realizada para esta revisão foi uma inspeção estática dos controladores, modelos, configurações e chamadas do cliente, seguida da comparação com o enunciado [1, 2]. Essa análise confirma a existência das rotas descritas, mas não comprova o comportamento em execução. Não foi localizado um projeto dedicado de testes unitários ou de integração. A collection Postman foi verificada como ficheiro JSON, mas os pedidos não foram executados contra os serviços. Os endpoints /health estão implementados; o estado dos serviços Render não foi confirmado. Para completar a evidência de entrega, devem ser executados testes no ambiente de destino e anexados os resultados, incluindo falhas e condições de erro.

7. Limitações e trabalho futuro

O protótipo atual tem lacunas importantes face ao enunciado. A especificação estática CSharp/openapi.yaml documenta as rotas atuais, mas a aplicação não gera nem serve OpenAPI em runtime. As APIs não aplicam autenticação e autorização por token; também não foi encontrada integração externa de negócio nem projeto de testes automatizados ou relatório de execução. A collection Postman facilita pedidos manuais, mas não prova que tenham sido executados com sucesso. A disponibilidade do Render também não foi verificada nesta revisão.

Há ainda diferenças entre a interface e o comportamento do backend: a atualização de membro ignora alguns campos aceites pelo DTO; documentos guardam um caminho mas não fazem upload; eventos não têm uma operação dedicada de presenças; e cargos não condicionam ações no servidor. Também não se encontraram multi-tenancy, checklists, notificações, orquestração Kubernetes, ensaios de carga ou estratégia de resiliência. Estes pontos devem ser apresentados como trabalho futuro, não como funcionalidades concluídas.

8. Conclusão

A versão existente em CSharp demonstra uma arquitetura distribuída com quatro APIs de negócio ASP.NET Core, um Gateway YARP, um cliente .NET MAUI, persistência MongoDB separada por domínio, Dockerfiles e configuração para publicação no Render. O cliente usa as operações de login e CRUD disponíveis, e a collection Postman reúne os pedidos que utiliza.

A comparação com o enunciado identifica requisitos ainda por cumprir ou provar: autorização das APIs, integração externa de negócio, testes executados e documentados e confirmação do funcionamento cloud. O contrato OpenAPI está documentado em CSharp/openapi.yaml, embora sem geração ou endpoint runtime. O relatório distingue estes factos e evita atribuir à implementação JWT, IPMA, Docker Compose, chamadas entre microserviços ou testes não evidenciados na pasta CSharp.

Bibliografia

[1] Universidade Politécnica do Cávado e do Ave. Arquiteturas e Integração de Sistemas: Enunciado do Trabalho Prático, revisão 1. Documento fornecido para esta revisão, 2026.

[2] TunaEngine. Código-fonte e documentação do protótipo na pasta CSharp: serviços Members, Events, Documents e Inventory; Gateway; cliente .NET MAUI; MICROSERVICES_RENDER.md, TunaEngine.postman_collection.json, openapi.yaml e OPENAPI.md. Consulta em 3 de novembro de 2026.

[3] Microsoft. Create web APIs with ASP.NET Core. https://learn.microsoft.com/en-us/aspnet/core/web-api/?view=aspnetcore-10.0 (consulta em 3 de novembro de 2026).

[4] Microsoft. YARP Configuration Files. https://learn.microsoft.com/en-us/aspnet/core/fundamentals/servers/yarp/config-files (consulta em 3 de novembro de 2026).

[5] MongoDB. MongoDB .NET/C# Driver Documentation. https://www.mongodb.com/docs/drivers/csharp/current/ (consulta em 3 de novembro de 2026).

[6] Microsoft. What is .NET MAUI? https://learn.microsoft.com/en-us/dotnet/maui/what-is-maui (consulta em 3 de novembro de 2026).

[7] Microsoft. Overview of OpenAPI support in ASP.NET Core API apps. https://learn.microsoft.com/en-us/aspnet/core/fundamentals/openapi/overview?view=aspnetcore-10.0 (consulta em 3 de novembro de 2026).